

- Description

This module manages an online course enrollment system, ensuring that:

1. **Students can register for courses as long as seats are available.**
2. **If a course reaches full capacity, students are placed on a waiting list.**
3. **When a student drops a course, the next student on the waiting list is enrolled automatically.**
4. **A log of all enrollments, cancellations, and waiting list actions** is maintained for administrative tracking**.

This system ensures fair enrollment management while preventing overbooking.

- Specification

module OnlineCourseEnrollment

var capacity: integer

var enrolledStudents: sequence of string

var waitingList: sequence of string

var enrollmentLog: sequence of string

process enrollStudent(studentName: string) flag: boolean

functional scenario form:

$|enrolledStudents| < capacity \ \&\& \ flag = true \ and \ enrolledStudents = enrolledStudents \oplus studentName$ and $enrollmentLog = enrollmentLog \oplus ("Enrolled: " + studentName)$, Student enrolled successfully.

||

$|enrolledStudents| \geq capacity \ \&\& \ flag = false \ and \ waitingList = waitingList \oplus studentName$ and $enrollmentLog = enrollmentLog \oplus ("Waiting list: " + studentName)$, Course full! Student added to waiting list.

end process

process dropStudent(studentName: string)

functional scenario form:

$(studentName \in enrolledStudents) \ \&\& \ enrolledStudents = remove(enrolledStudents, studentName) \wedge enrollmentLog = enrollmentLog \oplus "Student dropped"$ $\wedge$ (if $|waitingList| > 0$ then $enrolledStudents = enrolledStudents \oplus first(waitingList)$), Student dropped, waiting list updated.

OR

$(studentName \notin enrolledStudents) \ \&\& \ Student \ not \ enrolled$, no action taken.

end process

end module

- Java Code

```
package code;
import java.util.*;
class Student16 {
    private String name;
```

```

    public Student16(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}

class Course {
    private String courseName;
    private int capacity;
    private Set<String> enrolledStudents;
    private Queue<String> waitingList;
    private List<String> enrollmentLog;
    public Course(String courseName, int capacity) {
        this.courseName = courseName;
        this.capacity = capacity;
        this.enrolledStudents = new HashSet<>();
        this.waitingList = new LinkedList<>();
        this.enrollmentLog = new ArrayList<>();
    }
    public void enrollStudent(String studentName) {
        if (enrolledStudents.size() < capacity) {
            enrolledStudents.add(studentName);
            enrollmentLog.add(String.format("Enrolled: %s in %s", studentName, courseName));
            System.out.printf("Enrollment successful! %s registered for %s.%n", studentName, courseName);
        } else {
            waitingList.add(studentName);
            enrollmentLog.add(String.format("Waiting list: %s added to %s", studentName, courseName));
            System.out.printf("Course full! %s added to the waiting list for %s.%n", studentName, courseName);
        }
    }
    public void dropStudent(String studentName) {
        if (enrolledStudents.contains(studentName)) {
            enrolledStudents.remove(studentName);
            enrollmentLog.add(String.format("Dropped: %s from %s", studentName, courseName));
            System.out.printf("Student %s dropped from %s.%n", studentName, courseName);
        }
        if (!waitingList.isEmpty()) {
            String nextStudent = waitingList.poll();

```

```

        enrolledStudents.add(nextStudent);
        enrollmentLog.add(String.format("Moved from waiting list: %s enrolled in %s", nextStudent, courseName));
        System.out.printf("Next in line: %s is now enrolled in %s.%n", nextStudent, courseName);
    }
} else {
    System.out.printf("Student %s is not enrolled in %s.%n", studentName, courseName);
}
}
public void printEnrollmentLog() {
    System.out.println("\nEnrollment Log for " + courseName + ":");
    for (String log : enrollmentLog) {
        System.out.println(log);
    }
}
}
public class c16_OnlineCourseSystem {
    public static void main(String[] args) {
        Course javaCourse = new Course("Java Programming", 2);
        // testcase-VT:
        // javaCourse.enrollStudent("Alice");
        // javaCourse.enrollStudent("Bob");
        // javaCourse.enrollStudent("Charlie");
        // javaCourse.enrollStudent("David");
        // javaCourse.dropStudent("Bob");
        // javaCourse.dropStudent("Eve");
        // javaCourse.dropStudent("Alice");
        // javaCourse.dropStudent("David");

        // testcase-FT:
        // javaCourse.enrollStudent("??");
        // javaCourse.enrollStudent("safh2");
        // javaCourse.enrollStudent("");
        // javaCourse.enrollStudent("e33-d");
        // javaCourse.dropStudent("??w?");
        // javaCourse.enrollStudent("jio");
        // javaCourse.enrollStudent(" ");
        // javaCourse.enrollStudent("u4D-d");
        // testcase-Z3:
        // javaCourse.enrollStudent("Charlie");
        // javaCourse.enrollStudent("Bob");
        // javaCourse.enrollStudent("");
    }
}

```

```
//      javaCourse.enrollStudent("e33-d");
//      javaCourse.dropStudent("??w?");
//      javaCourse.enrollStudent("jio");
//      javaCourse.enrollStudent(" ");
//      javaCourse.enrollStudent("u4D-d");

// testcase-UVT:
javaCourse.enrollStudent("Charlie");
javaCourse.enrollStudent("safh2");
javaCourse.enrollStudent("");
javaCourse.enrollStudent("e33-d");
javaCourse.dropStudent("Bob");
javaCourse.dropStudent("Charlie");
javaCourse.enrollStudent("David");
javaCourse.enrollStudent("David");
javaCourse.printEnrollmentLog();
    }
}
```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	1.0 %	8	834	842
✓ src	1.0 %	8	834	842
✓ code	1.0 %	8	834	842
✓ c16_OnlineCourseSystem.java	100.0 %	8	0	8
> c16_OnlineCourseSystem		0	0	0
> Course	100.0 %	8	0	8

3. Vibration testing instruction coverage: 95.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.6 %	265	16,008	16,273
✓ src	1.6 %	265	16,008	16,273
✓ code	1.6 %	265	16,008	16,273
✓ c16_OnlineCourseSystem.java	95.7 %	265	12	277
> c16_OnlineCourseSystem	91.7 %	33	3	36
> Course	100.0 %	232	0	232

4. Fuzz testing branch coverage: 62.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	837	842
✓ src	0.6 %	5	837	842
✓ code	0.6 %	5	837	842
✓ c16_OnlineCourseSystem.java	62.5 %	5	3	8
> Course	62.5 %	5	3	8
> c16_OnlineCourseSystem		0	0	0

5. Fuzz testing instruction coverage: 65.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.1 %	181	16,092	16,273
✓ src	1.1 %	181	16,092	16,273
✓ code	1.1 %	181	16,092	16,273
✓ c16_OnlineCourseSystem.java	65.3 %	181	96	277
> Course	63.8 %	148	84	232
> c16_OnlineCourseSystem	91.7 %	33	3	36

6. Z3 branch coverage: 37.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.4 %	3	837	840
✓ src	0.4 %	3	837	840
✓ code	0.4 %	3	837	840
✓ c16_OnlineCourseSystem.java	37.5 %	3	5	8
> Course	37.5 %	3	5	8
> c16_OnlineCourseSystem		0	0	0

7. Z3 instruction coverage: 57.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.7 %	158	21,935	22,093
✓ src	0.7 %	158	21,935	22,093
✓ code	0.7 %	158	21,935	22,093
✓ c16_OnlineCourseSystem.java	57.5 %	158	117	275
> Course	54.7 %	127	105	232
> c16_OnlineCourseSystem	91.2 %	31	3	34

8. Unimproved V-method branch coverage: 87.5%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.8 %	7	833	840
✓ src	0.8 %	7	833	840
✓ code	0.8 %	7	833	840
✓ c16_OnlineCourseSystem.java	87.5 %	7	1	8
> Course	87.5 %	7	1	8
> c16_OnlineCourseSystem		0	0	0

9. Unimproved V-method instruction coverage: 95.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.2 %	265	21,830	22,095
✓ src	1.2 %	265	21,830	22,095
✓ code	1.2 %	265	21,830	22,095
✓ c16_OnlineCourseSystem.java	95.7 %	265	12	277
> c16_OnlineCourseSystem	91.7 %	33	3	36
> Course	100.0 %	232	0	232